# PROYECTO INTEGRADOR

**Asignatura: Programación IV**

**Carrera: Ingeniería en Informática**

**Docente: Prof. Marcos Arrieta**

## 🏭 1. DESCRIPCIÓN Y OBJETIVO GENERAL

El proyecto integrador consiste en la planificación, gestión, diseño, desarrollo y despliegue de una solución web empresarial orientada a resolver una problemática real de una **PyME o Empresa localizada en Paraguay**.

La solución técnica estará desarrollada en **Laravel** con conexión a base de datos relacional (**MySQL / PostgreSQL**), orquestada mediante contenedores con **Docker / Portainer**, e integrada con **HAProxy** como balanceador de carga frontal para distribuir el tráfico entre **dos arquitecturas web distintas**.

## 📋 2. METODOLOGÍA DE TRABAJO Y DOCUMENTACIÓN OBLIGATORIA

Cada equipo deberá adoptar obligatoriamente una metodología de trabajo ágil (**Scrum** o **Kanban**) para la gestión del proyecto durante el semestre.

### A. Herramientas de Gestión Exigidas:

- Uso obligatorio de un tablero ágil digital (**Jira, Trello, GitHub Projects o Azure DevOps**).

### B. Documentación Metodológica Entregable:

Según la metodología seleccionada, los grupos deberán elaborar y mantener actualizada la siguiente documentación:

1. **Gestión con Scrum:**
    - **Product Backlog:** Lista priorizada de Historias de Usuario (User Stories) con criterios de aceptación claros.
    - **Sprint Backlog & Planning:** Planificación de Sprints (Sprints de 2 a 3 semanas).
    - **Evidencia de Ceremonias:** Registro/actas de *Daily Standups* breves, *Sprint Review* y *Sprint Retrospective* al finalizar cada hito.
2. **Gestión con Kanban:**
    - **Tablero Kanban:** Definición clara de columnas de flujo (*To Do, In Progress, Code Review/QA, Done*).
    - **Límites de Trabajo en Proceso (WIP Limits):** Restricciones de tareas simultáneas por columna para evitar cuellos de botella.
    - **Métricas de Flujo:** Diagrama de Flujo Acumulado (*Cumulative Flow Diagram*) o seguimiento de *Lead Time* y *Cycle Time*.

# 🎯 3. DEFINICIÓN DEL ALCANCE Y MODELO DE DATOS (MER)

## A. Alcance del Proyecto (Scope Definition):

Cada equipo deberá redactar y entregar la **Declaración de Alcance (*Project Scope Statement*)**, la cual incluirá:

- **Objetivos del Negocio:** Justificación del impacto de la solución en la PyME paraguaya seleccionada.
- **Límites del Proyecto (In-Scope / Out-of-Scope):** Definición explícita de qué funcionalidades **sí incluye** el sistema y qué aspectos quedan fuera de este semestre para evitar la corrupción del alcance (*Scope Creep*).
- **Entregables Principales:** Módulos funcionales, API REST, despliegue en Portainer, balanceo con HAProxy y documentación.

## B. Modelo Entidad-Relación (MER):

Diseño estructurado de la persistencia de datos del negocio:

- **Diagrama MER / DER:** Presentación del diagrama conceptual y lógico de la base de datos relacional.
- **Diccionario de Datos:** Detalle de tablas, campos, tipos de datos, llaves primarias ($PK$), llaves foráneas ($FK$), restricciones e índices de optimización.
- **Normalización:** Justificación de la estructura en **Tercera Forma Normal (3FN)**.

# 🇵🇾 4. CASO DE USO EMPRESARIAL (PyMEs de Paraguay)

Ejemplos de rubros del entorno local sugeridos:

- **Retail / Comercial:** Gestión de stock, ventas y facturación para minimarkets o tiendas.
- **Servicios / Logística:** Rastreos de envíos, gestión de paquetes o agendamiento de turnos.
- **Gastronomía:** Comandera digital, gestión de pedidos e inventarios para restaurantes.
- **Fintech / Cobros:** Módulo de gestión de cobros e integración con pasarelas de pago locales.

# 👥 5. ROLES DENTRO DEL EQUIPO

Cada equipo (3 a 4 integrantes) asignará formalmente los siguientes roles:

1. **Scrum Master / Project Manager:** Responsable de la metodología ágil, gestión del tablero, control del alcance y facilidades del equipo.
2. **Backend & Software Architect Lead:** Responsable de la lógica en Laravel, arquitectura de datos, migraciones, Eloquent/PDO y APIs.

3. **DevOps & Cloud Engineer:** Responsable de Docker, docker-compose.yml, Stack en **Portainer** y archivo .cfg de **HAProxy**.
4. **Frontend & QA Engineer:** Responsable de las vistas/interfaz, consumo de endpoints, pruebas funcionales y *benchmarking*.

# 🛠️ 6. REQUISITOS TÉCNICOS E INFRAESTRUCTURA

1. **Framework & BD:** Laravel (v10 o v11+) conectado a MySQL o PostgreSQL.
2. **Persistencia & Sesiones:** Redis o BD centralizada para evitar pérdida de sesión al balancear entre nodos.
3. **Balanceador de Carga (HAProxy):**
    - Algoritmos de balanceo (roundrobin o leastconn).
    - Dashboard de métricas activo (**HAProxy Stats** en puerto 8404).
    - Verificación de estado (*Health Checks*) y *Failover* automático.
4. **Gestión de Contenedores:** Despliegue del Stack mediante **Portainer**.

# 🏛️ 7. ARQUITECTURAS COMPARATIVAS (Asignación por Grupo)

- Pendiente por definir

# 📅 8. CALENDARIO DE ENTREGAS Y PRE-REQUISITOS OBLIGATORIOS

⚠️ **REQUISITO INDISPENSABLE:** La presentación y aprobación de cada uno de los avances en las fechas estipuladas es **pre-requisito obligatorio** para la habilitación a la **Defensa Final**.

| **Hito / Avance** | **Fecha Límite** | **Entregables Requeridos** |
|---|---|---|
| **Avance 1** | **28 de Agosto** | • Definición del Alcance (*Project Scope*) y caso PyME.<br><br>• Configuración de metodología (Tablero Jira/Trello y Product Backlog). |

| | | |
|---|---|---|
| | | • Diagrama MER / DER y Diccionario de Datos.<br><br>• Entorno base Docker (Laravel + MySQL local). |
| **Avance 2** | **25 de Septiembre** | • Reporte de Sprints / Flujo Kanban acumulado.<br><br>• Módulo CRUD principal funcional con reglas de negocio.<br><br>• Manejo de sesiones centralizadas (Redis/BD).<br><br>• Despliegue inicial del Stack en **Portainer**. |
| **Avance 3** | **30 de Octubre** | • Evidencia de ceremonias ágiles y actualización del tablero.<br><br>• Configuración e integración del balanceador **HAProxy**. |

| | | |
|---|---|---|
| | | • Despliegue de la Arquitectura A y B activas.<br><br>• Demostración de *Failover* (tolerancia a fallos) en vivo. |
| **Avance 4** | **27 de Noviembre** | • Cierre metodológico (Sprint Review/Retrospective final o Métricas de flujo).<br><br>• Pruebas de rendimiento (*Benchmarking* RPS/Latencia Arq A vs B).<br><br>• Dashboard **HAProxy Stats** (Puerto 8404) con tráfico real.<br><br>• Repositorio Git y documentación técnica/metodológica completa. |